Functional programming on Filter and reduce

Thursday, September 18, 2025 9:52 PM

write a program to get the following output

Input--->s='hai helloo how'
Output--->['hi','ho','hw']

```
s='hai helloo how'.split()#['hai','helloo','how']
p=lambda n:n[0]+n[-1]
l=map(p,s)
print(list(l))

#or
print(list(map(lambda n:n[0]+n[-1],input('enter the string:').split())))
```

write a program to get the following output

```
#input:range(1,6)
#output={1:1,2:8,3:27,4:64,5:125}#{1:cubeof the number,2:'cube of the number'}

cube=lambda n:(n,n**3)
out=map(cube,range(1,6))
print(dict(out))

#or

print(dict(map(lambda k:(k,k**3),range(int(input('enter the starting
value:')),int(input('enter the ending value:'))))))
```

write a program to get the following output

```
#s='programs on map function'
#output={'programs':'smargorp','on':'no','map':'pam','function':'noitcnuf'}

s='programs on map function'.split()#['programs','on','map','function']
rev=lambda l:(l,l[::-1])
result=map(rev,s)
print(dict(result))
```

```
#or

print(dict(map(lambda l:(l,l[::-1]),'programs on map function'.split())))


#set

sqr=lambda a:a**2
sqr_value=(map(sqr,{10,20,30,40,50,10,20})
print(set(sqr_value))
```

filter():

'''filter is a inbuilt function which is used to extract only the required values from the collection'''

syntax of filter():

```
variable_name=filter(function_name,collection)
print(list(variable_name))

#or

print(list(filter(function_name,collection)))
```

1.wap program to extract even numbers between the given range 1 to 50

```
even=lambda n:n%2==0
even_num=filter(even,range(1,50))#1,2,3,4.....50
print(list(even_num))
```

2.wap to extract all the strings data items from the tuple only if it starting with lowercare character and ending with uppercase.

```
extract=lambda v:type(v)==str and 'a'<=v[0]<='z' and 'A'<=v[-1]<='Z'
out=filter(extract,eval(input('enter the tuple data:')))
```

```
print(tuple(out))
```

Or

```
print(tuple(filter(lambda v:type(v)==str and 'a'<=v[0]<='z' and 'A'<=v[-1]
<='Z,eval(input('enter the tuple data:')))))
```

3.**wap to find the square all the even numbers from the given List only if it is integer.**

```
even=lambda n:type(n)==int and n%2==0
out=filter(even,[12,3.4,5.5,7+8j,2,94,33,'10','15'])
#print(list(out))#[12, 2, 94]
sqr=lambda d:d**2
res=map(sqr,out)
print(list(res))
#sqr=lambda d:d**2
#print(list(map(sqr,out)))
```

Or

```
print(list(map(lambda d:d**2,filter(lambda n:type(n)==int and n%2==
0,[12,3.4,5.5,7+8j,2,94,33,'10','15']))))'''
```

4.wap to extract all the collection data or values present in a list which has even length

```
l=[10,20,'hi',[10,20],(5.5,6+5j),'pythonn',{100,200,100,400}]
extract=lambda n:type(n) in [str,list,set,tuple,dict] and len(n)%2==0
res=filter(extract,l)
print(list(res))

#or

print(list(filter(lambda n:type(n) in [str,list,set,tuple,dict] and len(n)%2==
0,[10,20,'hi',[10,20],(5.5,6+5j),'pythonn',{100,200,100,400}])))
```

#Assignment--->22-08-25

5.wap to extract all the prime numbers between 2 to 50
2--->2,1
3--->3,1
4--->1,2,4

#wap to extract all the number from a collection(list,set,tuple) only if it divisible by 5
input:[10,20,'hi',[10,20],(5.5,6+5j),'pythonn',{100,200,100,400}]

#wap to extract only veg items from given collection
input=['pizza','chicken','mutton','fish','burger','paneer']
output=['pizza','burger','paneer']

#Assignment--->22-08-25
#wap to extract all the prime numbers between 2 to 50
2--->2,1
3--->3,1
4--->1,2,4

```
def is_prime(n):
    if n<1:#n<=1 or n<2
        return False

    for i in range(2,n):
        if n%i==0:
            return False
    return True
print(list(filter(is_prime,range(0,20))))
```

#wap to get whose marks greater than 35
marks={'kesavan':35,'pavithra':75,'prashant':80,'srinivas':25}

```
l=lambda k: k[1]>=35
n=filter(l,marks.items())
print(dict(n))
```

or

```
print(dict(filter(lambda k: k[1]>=35,marks.items())))
```

wap program to check both collections are pointing same or different address.

```
a=lambda m,n:'poiting same address' if id(m)==id(n) else 'pointing different addres'
print(a(eval(input('enter the data')),eval(input('enter the data2'))))
```

wap to extract all the number from a collection(list,set,tuple) only if it divisible by 5

input:[10,20,'hi',[10,20],(5.5,6+5j),'pythonn',{100,200,100,400}]

Logic-->lambda n:type(n) and n%5==0

wap to extract only veg items from given collection

input=['pizza','chicken','mutton','fish','burger','paneer']
output=['pizza','burger','paneer']

veg=['pizza','burger','paneer']
Logic--->lambda n:n in ['pizza','burger','paneer']

wap program to extract all the even numbers and sind the square of those values.

```
even = lambda n: type(n) == int and n % 2 == 0
out = filter(even, [12, 3.4, 5.5, 7+8j, 2, 94, 33, '10', '15', [10,20]])
out_list = list(out)
print((out_list))   # [12, 2, 94]
sqr = lambda d: d**2
res = list(map(sqr, out_list))
print(res)
```

reduce():

it is a inbuilt function which used to perform the operation in betwee values or element present inside collection.

```
#how do you find the sum?
[1,2,3,4,5,6]=21

s=0#sum
for i in [1,2,3,4,5,6]:
    s=s+i
print(s)
```

syntax of reduce()

```
from functools import reduce
variable name=(functioname,collection)
print(variable name)
```

```
from functools import reduce

ADD=lambda s,i:s+i
out=reduce(ADD,[1,2,3,4,5,6])
print(out)


#or

print(reduce(lambda s,i:s+i,[1,2,3,4,5,6]))#mul,sub,division
printreduce(lambda s,i:s-i,[1,2,3,4,5,6]))
print(reduce(lambda s,i:s*i,[1,2,3,4,5,6]))
print(reduce(lambda s,i:s/i,[1,2,3,4,5,6]))
```

find the maximum number in collection

```
n=[75,96,32,5,99,2,27]
l=lambda a,b:a if a<b else b
```

```
out=reduce(l,n)
print(out)

#or

print(reduce(lambda a,b:a if a<b else b,[75,96,32,5,99,2,27]))
print(reduce(lambda a,b:a if a<b else b,['python','code','cat']))
print(reduce(lambda a,b:a if a<b else b,[[10,20],[30,40]]))

#a=75,b=96--->75>96--->96
#a=96,b=32--->96>32--->96
#a=96,b=5---->96>5--->96
#a=96,b=99--->96>99---->99
```

filter()

Q1. What is the difference between map() and filter()?

- map() applies a function to every element and transforms the list.
- filter() applies a function and **keeps only the elements where the function returns True**.

```
nums = [1, 2, 3, 4, 5]
print(list(map(lambda x: x*2, nums)))   # [2, 4, 6, 8, 10]
print(list(filter(lambda x: x%2==0, nums)))  # [2, 4]
```

Q2. Why does filter() return a filter object instead of a list?

- Because filter() is a **lazy iterator**. It doesn't compute everything at once.
- To see results: list(filter(...)) or tuple(filter(...)).

Q3. Can I use filter() without lambda?

Yes, you can pass a normal function.

```
def is_even(n):
   return n % 2 == 0
nums = [1, 2, 3, 4, 5, 6]
print(list(filter(is_even, nums)))  # [2, 4, 6]
```

Q4. What happens if no elements satisfy the condition?
You'll get an **empty filter object** → [] after converting to list.

```
nums = [1, 3, 5]
print(list(filter(lambda x: x%2==0, nums)))  # []
```

Q5. What if the function always returns True or always False?
Always True → keeps all elements.
Always False → removes all elements.

```
nums = [1, 2, 3]
print(list(filter(lambda x: True, nums)))   # [1, 2, 3]
print(list(filter(lambda x: False, nums)))  # []
```

Q6. Example real use?
Filter words of length > 5.

```
words = ["apple", "banana", "cherry", "kiwi", "mango"]
print(list(filter(lambda w: len(w) > 5, words)))
# ['banana', 'cherry']
```

reduce():

Q1. What is the difference between reduce() and a loop?
Both combine elements into a single result.
reduce() does it functionally, while a loop is imperative.

```
from functools import reduce
nums = [1, 2, 3, 4]
# using reduce
print(reduce(lambda x, y: x+y, nums))  # 10
# using loop
s = 0
for n in nums:
    s += n
print(s)  # 10
```

Q2. Why is reduce() in functools and not built-in?
Because in Python 3, it was moved out of built-ins to keep the core smaller.
It's not used as often as map and filter.

Q3. Why does reduce() return a single value while map() and filter() return collections?
Because reduce() **reduces** multiple values into **one value** (sum, product, gcd, etc.).

Q4. How can I use reduce() for sum, product, or max?

```
from functools import reduce
nums = [1, 2, 3, 4]
# sum
print(reduce(lambda x, y: x+y, nums))  # 10
# product
print(reduce(lambda x, y: x*y, nums))  # 24
# maximum
print(reduce(lambda x, y: x if x>y else y, nums))  # 4
```

Q5. What happens if the list is empty?
Without an initial value → error.
With an initial value → returns that initial value.

```
nums = []
print(reduce(lambda x, y: x+y, nums, 0))  # 0
```

Q6. What if I don't provide an initial value?
reduce() takes the **first two elements** as a starting point.

```
nums = [1, 2, 3]
print(reduce(lambda x, y: x+y, nums))  # 6
# internally: ((1+2)+3)
```

Q7. How is reduce(lambda x, y: x+y, numbers) different from sum(numbers)?
Both give the same result, but sum() is faster and more readable.
reduce() is useful for custom logic (not just sum).

Q8. Can I use reduce() to flatten a list of lists?

Yes!

```
lists = [[1, 2], [3, 4], [5, 6]]
flat = reduce(lambda x, y: x+y, lists)
print(flat)  # [1, 2, 3, 4, 5, 6]
```